Cool Project

We cooked with Toeplit matrices today following Martha Stewart

Ali Arda BARUT
Rafael LLAMAS

March 16, 2026

Contents

1 Introduction **2**

2 Definitions **2**
2.1 Toeplitz Matrices . . . 2
2.1.1 Quasi Toeplitz Matrices . . . 2
2.2 Displacement Matrices . . . 2
2.2.1 Mathematical Approach . . . 3
2.2.2 Computational Approach . . . 3
2.3 Generator Matrices . . . 3
2.4 Displacement Operators . . . 4
2.5 Conclusion & Implementation . . . 4

3 Operation **4**
3.1 Addition . . . 4
3.1.1 Exemple . . . 5
3.2 Multiplication . . . 5
3.2.1 Exemple . . . 5
3.3 Inversion . . . 5

1 Introduction

Hello hello welcome!

2 Definitions

In this section, we will discuss what are Toeplitz Matrices, how are they defined and what are their properties such that we can take advantage during matrix implementations.

2.1 Toeplitz Matrices

A Toeplitz matrix is in which each element in first row and first column descends diagonally. For a matrix $A \in \mathbb{K}^{n\times m}$, it satisfies the condition $A_{i,j} = A_{i+1,j+1}$. Formally, it is defined by a sequence of elements $\{a_k\}$ such that:

$$A = \begin{pmatrix} a_0 & a_{-1} & a_{-2} & \cdots & a_{-(m-1)} \\ a_1 & a_0 & a_{-1} & \cdots & a_{-(m-2)} \\ a_2 & a_1 & a_0 & \cdots & a_{-(m-3)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n-1} & a_{n-2} & a_{n-3} & \cdots & a_{n-m} \end{pmatrix} \tag{1}$$

$$A_{i,j} = a_{i-j}, \quad 0 \le i < n,\ 0 \le j < m$$

2.1.1 Quasi Toeplitz Matrices

A Quasi-Toeplitz matrix is not strictly Toeplitz but shares similar structural properties. We can also define Quasi-Toeplitz matrix as a Toeplitz Matrix that has noise within. Notice that we can still benefit the structural advantages of Toeplitz partially in these matrices.

2.2 Displacement Matrices

For a matrix $A \in \mathbb{K}^{n\times m}$, we define the displacement matrix ∇A using:

$$\nabla A = A - ZAZ^T \tag{2}$$

where Z represents the lower shift matrix and Z^T represents the upper shift matrix. Essentially, Z is a matrix with ones on the first sub-diagonal and zeros everywhere else:

$$Z = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix} \quad Z^T = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix} \tag{3}$$

Multiplying a matrix A by Z from the left (ZA) shifts the rows of A down by one (zeros at the top). Multiplying by Z^T from the right (AZ^T) shifts the columns to the right, (zeros at the first column). Resulting in an operation ZAZ^T that shifts the entire matrix A one step to the bottom-right.

2.2.1 Mathematical Approach

For a generic Toeplitz matrix of size $n \times m$, this subtraction results in zeros everywhere except for the first row and the first column.

$$\nabla A = \begin{pmatrix} y_0 & x_1 & x_2 & \cdots & x_{m-1} \\ y_1 & 0 & 0 & \cdots & 0 \\ y_2 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ y_{n-1} & 0 & 0 & \cdots & 0 \end{pmatrix} \in \mathbb{K}^{n \times m} \tag{4}$$

This allows us to represent the $n \times m$ matrix using only $O(n + m)$ elements rather than storing the full $O(nm)$ dense matrix.

2.2.2 Computational Approach

While mathematical definition involves matrix multiplication, in this case $O(n^2 * m)$ or similar depending on the multiplication method used, calculating ∇A based on matrix multiplication is inefficient.

The displacement calculation can be reduced to an element based loop with complexity $O(nm)$:

$$(\nabla A)_{i,j} = \begin{cases} A_{i,j} & \text{if } i = 0 \text{ or } j = 0 \\ A_{i,j} - A_{i-1,j-1} & \text{otherwise} \end{cases} \tag{5}$$

This approach allows us to iterate from the bottom row to the top updating values in place.

2.3 Generator Matrices

Displacement matrices, ∇A, typically have a low rank α, we call this displacement rank. For Toeplitz matrices, α is generally small ($\alpha \leq 2$). This allows us to store efficiently the displacement matrix into two smaller matrices G and H.

While the product $G \times H^T$ results in the displacement ∇A, the original matrix A can be reconstructed using the columns of the generators mentioned at Proposition 10.5 in [1]:

$$A - ZAZ^T = GH^T \quad \text{iff} \quad A = \sum_{i=1}^{\alpha} L(x_i)U(y_i), \tag{6}$$

where x_i (and thus y_i) are the columns of the generator G (and thus H), where for a column vector $v = [v_0 \dots v_{n-1}]^T$, $L(v)$ denotes the lower triangular Toeplitz matrix:

$$L(v) = \begin{pmatrix} v_0 & 0 & \dots & 0 \\ v_1 & v_0 & \dots & 0 \\ \vdots & \ddots & \dots & \vdots \\ v_{n-1} & \dots & v_1 & v_0 \end{pmatrix}$$

and $U(v)$ denotes the upper triangular Toeplitz matrix $L(v)^T$.

The matrix A can be written as a sum of elements of lower and upper triangular matrices:

$$A = \sum_{k=1}^{\alpha} L(g_k)U(h_k) \tag{7}$$

- $L(g_k)$ is a lower triangular Toeplitz matrix with the first column g_k.
- $U(h_k)$ is an upper triangular Toeplitz matrix with the first row h_k^T.

2.4 Displacement Operators

We define two displacements operators:

$$\begin{aligned} \phi_+(A) &= A - ZAZ^T = \nabla A \\ \phi_-(A) &= A - Z^T AZ \end{aligned} \tag{8}$$

In other words, $\phi_-(A)$ is the substraction of minuend A where subtrahend is the matrix A but with the columns shifted up and rows to right once.

Computationally, it follows the same path as the calculation of ∇A. Main difference being we can run this time in increasing order:

$$\phi_-(A)_{i,j} = \begin{cases} A_{i,j} & \text{if } i = n-1 \text{ or } j = m-1 \\ A_{i,j} - A_{i+1,j+1} & \text{otherwise} \end{cases} \tag{9}$$

Where `n` and `m` are respectfully the number of rows and columns of matrix A.

2.5 Conclusion & Implementation

We conclude the section of the definitions. Implementations of these concepts can be found in:

```
Project
\   src
|   \   displacement_matrices
|   |   |   displacement_matrices.c
|   |   |   displacement_matrices.h
```

gr_mat_displacement(gr_mat_t D, gr_mat_t A, disp_type_t type, gr_ctx_t ctx) Is the function taking as parameter the matrix A and returns into the matrix D, indicating destination, the displacement matrix of A. According to the type of displacement we pass via the parameter `type`, it calculates the implementations (5) or (9).

int gr_mat_G_H(gr_mat_t G, gr_mat_t H, gr_mat_t A, disp_type_t type, gr_ctx_t ctx)) Is the function that computes the generator matrices G and H^T for an input matrix A. Based on the displacement equations discussed previously, the equation (7), the function first calculates the displacement matrix according to the specified type of displacement. Since the displacement of a structured matrix (like a Toeplitz matrix) inherently results in a matrix with a low rank (α), D can be factored into two smaller matrices such that $D = GH^T$. The function achieves this by performing an LU decomposition on the displacement matrix D. It then extracts the independent columns to form G and the corresponding rows to form H^T. This is where the storage economy for structured matrices come from.

int gr_mat_reconstruct_A(gr_mat_t A, gr_mat_t G, gr_mat_t H, disp_type_t type, gr_ctx_t ctx)

3 Operation

3.1 Addition

Grace au générateurs, nous pouvons addtionner nos matrices sans utiliser la formule standart. En effet, selon la propriété 10.7 et 10.8 du cours (ref cours ICI), on apprends que la somme de 2 matrices toeplitz peut etre simplfié par la concatenation de leurs générateurs de déplacement. Cela permet de

passrr a un coup de O(nxm) a un cout de O(αxm) pour 2 matrices A et B de tailles nxm et leurs générateurs de taille α pour ceux de A et β pour ceux de B. Avec flint, on peut utiliser la fonction (ici la fonction) pour faire la concatenation de nos générateurs.

3.1.1 Exemple

On va prendre les matrices A, B et C le résultat de l'addition de A et B. (METTRE LES MATRICES ICI) si on prends les générateurs de A = (T,U) et B = (G,H) (ICI GÉNÉRATEURS) et que l'on fait la concaténation on obtiens (ICI RESULTAT DE LA CONCATENATION) pour verifier notre résultat on peut utiliser la fonction (ici la fonction) pour recomposer la matrice que l'on a obtenue D (ICI D)

On remarque bien que celle-ci est identique a la matrices C

3.2 Multiplication

Toujours avec les générateurs, on peut simplifier la multiplication de 2 matrices et donc réduire son cout en concatenant : - les générateurs de A=(T,U) et B=(G,H) - W = Z x A x Ztranspose x G - V = Btranspose x U - a (resp. b) un vecteur qui est la dernière colonne de Z x A (resp Ztranspose x B) cela nous permet de passer de O(n3) pour l'algo naif a O(α^2M(n))

3.2.1 Exemple

pareil donner des exemples de chaque etapes a bien rédiger

3.3 Inversion

Pour arda

References

[1] Alin Bostan, Frédéric Chyzak, Marc Giusti, Romain Lebreton, Grégoire Lecerf, Bruno Salvy, and Éric Schost. Algorithmes efficaces en calcul formel, August 2017. 686 pages. Édition 1.0.